Basic Lab5: Memory

Objective

This lab introduces memory management and control within hardware designs. You will implement a small circular memory system that automatically overwrites old data and tracks overwritten entries using dirty bits.

Design Problem

Design and implement a memory module that meets the following specifications:

- The memory contains 8 addresses, each 10 bits wide.
- Writing begins at address 0, and once all 8 addresses are used, writing wraps around to overwrite data starting again at address 0.
- When an address is overwritten, its corresponding dirty bit must be set to 1'b1.

Hint:
Implement counter(s) (as pointer(s)) to keep track of relevant information and manage wrap-around behavior within the memory.

Functional Description

1. Write Operation:
   Writing the input data to the memory with the control signal **we**.
   i. When **we==1'b1**, the input data (**din**) should be written into the memory once per clock cycle.
   ii. Writing starts at address 0, and continues sequentially.
   iii. When the last address (address 7) is reached, writing should **wrap around** to address 0, overwriting the existing data.
   iv. Each time an address is overwritten, its **dirty bit** should be set to 1'b1.
   v. When **we** transitions to **1'b0**, no further writes should occur.
2. Read Operation:
   Reading the data from the memory to the dout with the control signal **re**, based on the specified address (**addr**).
   i. When **re == 1'b1**, the data stored at the address specified by **addr** should be read and output on **dout** once per clock cycle.
   ii. When **re == 1'b0**, output should stop updating.
   iii. If the data at the read address has been overwritten previously, the output signal **dirty** must be set to **1'b1**; otherwise, **dirty** should be **1'b0**.
3. Reset Logic:
   i. The design uses a **synchronous active-high reset (rst)**.

ii. Upon reset, all memory contents are cleared to zero. **dout and dirty** are cleared to **0**. Internal counters or pointers are reset.

4. Clock Synchronization:
   i. The design must update at the **positive edge** of each clock cycle.
5. IO list:

Signal	I/O	Width	Description
clk	Input	1	Clock (positive edge triggered)
rst	Input	1	**Synchronous** active-high reset
we	Input	1	Write enable: write data to the memory when we==1'b1
re	Input	1	Read enable: read data from the memory when re==1'b1
addr	Input	3	Address (location) in memory to be read
din	Input	10	Input data to be written into memory
dirty	Output	1	Indicates whether the current address has been overwritten
dout	Output	10	Data output from the addressed memory location

6. Use the following template for your design:

```
module lab5_basic(
          input wire clk,
          input wire rst,
          input wire we,
          input wire re,
          input wire [2:0] addr,
          input wire [9:0] din,
          output reg dirty,
          output reg [9:0] dout
);
    // add your design here
endmodule
```

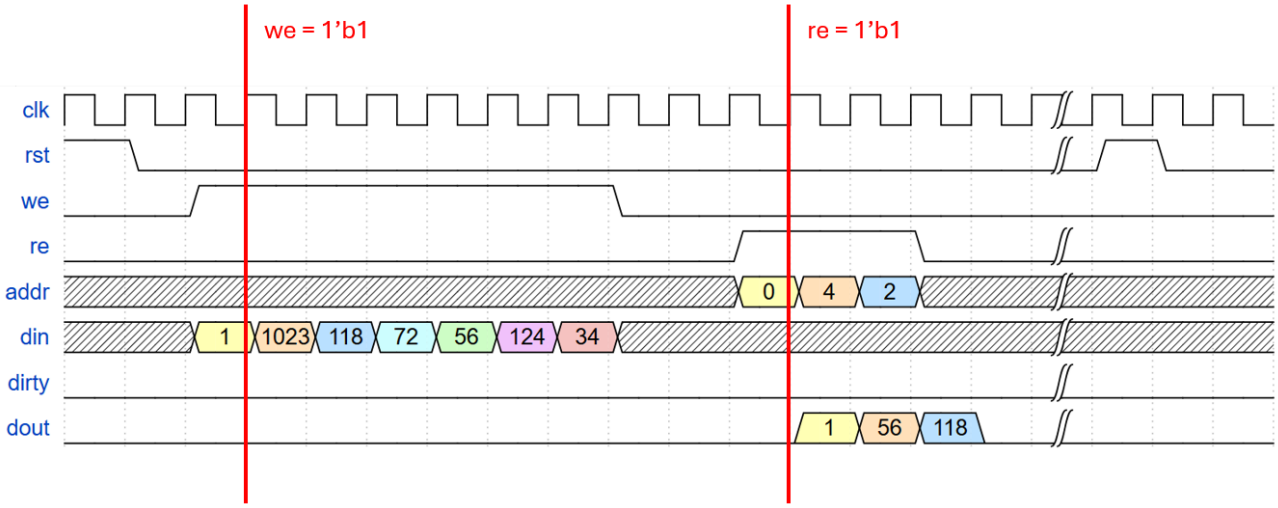
7. Grading:
   i. Score

Cases 1-4	20% each
Cases 5-6	10% each

   ii. **Note**: Your score is based on the **correctness of your solution**, not solely on the specific console messages from the testbench.

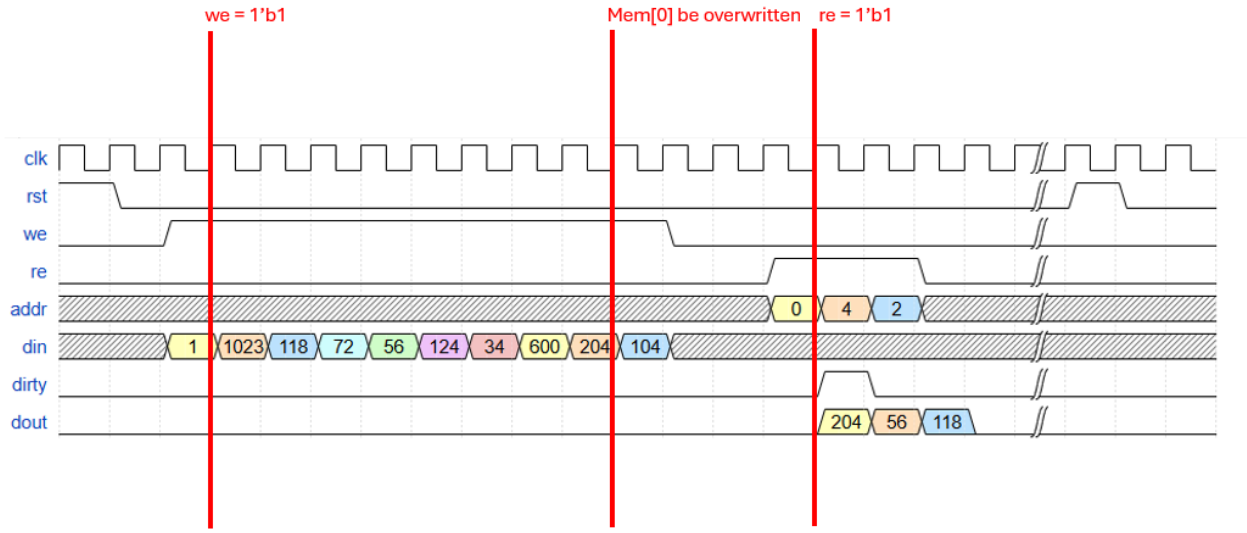
8. Waveform Reference:

Example 1:



In this example, when **we** is 1, data is written starting from address 0. When **re** is 1, the value from the corresponding address specified by address **(addr)** is read and output to **dout**.

Example 2:



In this example, addresses 0 and 1 have been overwritten before, while the other addresses have not. Therefore, when re = 1 and addr = 0, dirty = 1. When re = 1 and addr = 2 and 4, dirty = 0.

Attention

- Download the *.cpp files from OJ. Change them to *.zip and decompress them. (Skip the *.h files. The OJ enforces that there must be a *.h file.)
- Submit the Verilog file to OJ immediately once it is done.
- Please take the OJ password slip with you when leaving your seat. Do not litter!
- **Follow the instructions in "Lab5 Basic Simulation Guide.pdf" to run your simulation.**
- **Important**: DO NOT copy-and-paste code segments from the PDF materials, as this can introduce invisible non-ASCII characters that may cause hard-to-debug syntax errors.
- If you create several modules, merge them into a single Verilog file. Submit only one Verilog file, lab5_basic.v.
  - You have the responsibility to check if the submission is successful.
  - DO NOT submit ZIP files, which will be considered as incorrect format.
- **Do not modify** the testbench or any patterns.